MediQueue

Clinic Appointment Desk

DevOps Final Capstone — application, CI/CD, infrastructure,

Kubernetes, GitOps and observability

Khushi Sarawagi · 24BCS10361

Project Overview

- Clinic front desk for booking patient visits across six departments
- Reception books, checks in, completes or cancels appointments
- Doctors are a first-class resource; each appointment references one
- Live queue view: KPI strip, status tabs, search, pagination
- Built as an original domain for the DevOps capstone, not a TaskBoard clone

The Problem It Solves

- Paper diaries and phone-call scheduling lose track of who is next
- No shared answer to: how many patients are still waiting right now?
- MediQueue keeps one queue that the whole front desk can see
- Urgency is explicit, so the sickest patient is visible first
- Everything is an API call, so the same data feeds the dashboard

Tech Stack

- Frontend React 19, Vite, Lucide icons, hand-written responsive CSS
- API Python 3.13, FastAPI, Pydantic, SQLAlchemy 2
- Database PostgreSQL 17 with Alembic migrations
- Quality Pytest (26 cases), Ruff, Playwright browser regression suite
- Containers Non-root Alpine images, multi-stage Nginx frontend
- CI/CD GitHub Actions, Trivy gate, GHCR by commit SHA
- Infra Terraform (VPC + EKS), Helm, Kubernetes, Prometheus, Grafana

Application Data Model

- doctors id, full_name, department, specialization, active
- appointments id, patient_name, patient_phone, doctor_id (FK),
- scheduled_for, reason, status, urgency, notes
- Department belongs to the doctor, so it is never duplicated
- Statuses: scheduled, checked_in, completed, cancelled
- Urgency: routine, soon, urgent
- 11 API routes including full CRUD on both resources plus /api/stats

Testing And Code Quality

- Tests run before any image is built or pushed — the first quality gate
- 26 pytest cases across both resources and every route group
- Covered: CRUD, filters, stats, validation, 404/409, wildcard escaping,
- pagination stability, readiness failure, metric cardinality
- Local runs use isolated in-memory SQLite; CI uses dedicated PostgreSQL
- Ruff enforces import order, bugbear rules and a 110-column limit
- 17 Playwright checks exercise the real running application in a browser

Containers

- backend python:3.13-alpine, runs as UID 10001, pip removed from image
- frontend node:24 build stage, nginx-unprivileged runtime as UID 101
- docker compose up --build starts Postgres, a migration job, API, nginx
- The migrate service runs alembic upgrade head and must finish first
- Compose healthchecks gate each service on the previous one being ready

CI/CD Pipeline

- Job 1 TEST ruff, alembic, pytest on PostgreSQL, frontend build,
- helm lint, terraform validate (no cloud resources)
- Job 2 BUILD both images tagged with the full Git commit SHA
- SCAN Trivy on both images, exit-code 1 on HIGH/CRITICAL
- DEPLOY ephemeral Kind cluster, helm upgrade --install, smoke test
- VERIFY Playwright suite against the deployed Kubernetes app
- PUBLISH push to GHCR, then promote the SHA into values-release.json
- Commit A -> image A -> release A. Traceability from prod back to source.
- Result: run 38755163712 green in five minutes — 26 tests, 17 browser
- checks, both Trivy scans clean.

Security: Trivy Gate

- Scans installed OS packages and application dependencies in both images
- The pipeline fails on any HIGH or CRITICAL finding — no continue-on-error
- In CI both images scan clean: the Alpine layer and every Python package
- report zero vulnerabilities, so the gate is armed but never has to fire
- Security layers are different things: SAST, dependency scanning, secret
- scanning, container scanning and runtime security are not interchangeable
- A clean scan is not a proof of secure business logic or authorisation
- No credentials are committed; local secrets are generated at deploy time

Terraform: AWS Infrastructure

- VPC 10.52.0.0/16 across two availability zones
- Two public subnets and two private subnets, tagged for load balancers
- One NAT gateway for private subnet egress
- Managed EKS control plane with api/audit/authenticator logging
- Managed node group: 1-3 x t3.medium, AL2023, 30 GB disk
- EBS CSI driver via IRSA so PersistentVolumeClaims can bind
- EKS API restricted to supplied CIDRs; 0.0.0.0/0 is rejected by validation

Architecture

GitHub push	commit SHA is the artifact identity
GitHub Actions	ruff · pytest · frontend build · helm lint · terraform validate
Build and scan	docker build both images · Trivy HIGH/CRITICAL gate
Ephemeral Kind	helm upgrade --install · smoke test · Playwright suite
GHCR	images tagged with the full commit SHA
GitOps promotion	values-release.json -> Argo CD reconciles
Kubernetes	Ingress -> frontend / backend (/api) -> PostgreSQL PVC
Observability	Prometheus scrapes every replica -> Grafana dashboard
Terraform	VPC (2 AZ) + EKS managed node group for the AWS target

Kubernetes And Helm

- Helm renders 16 objects from one chart and one values file
- backend and frontend Deployments, two replicas each, non-root runtime
- ClusterIP Services; readiness gates endpoint membership
- PostgreSQL StatefulSet with a PersistentVolumeClaim
- A migration Job runs before the backend init container waits for schema
- Ingress routes /api to the backend and / to the frontend
- HPA targets 70% CPU across 2-4 backend replicas; 2 -> 3 observed under load

Observability

- The API exposes /metrics in Prometheus text format
- A headless Service lets Prometheus discover every backend replica
- Counters are labelled with route templates, never raw IDs
- Grafana ships with a provisioned MediQueue / Service Health dashboard:
- healthy replicas, request rate, p95 latency, route and status breakdown
- Alerts: backend unavailable and error ratio over five percent
- No external notification target is configured in the local demo

GitOps With Argo CD

- The Application tracks helm/mediqueue on main
- Promoted release first, local overlay last, so the cluster uses its own images
- CI writes values-release.json only after tests, scans and Kind smoke pass
- Automated sync and self-healing are on; automatic pruning is off
- Promotion commits use [skip ci] to avoid an image-build loop
- One deployment owner at a time: Helm for the Helm demo, Argo CD after

Troubleshooting Lab

- ImagePullBackOff: diagnose from pod events, fix the tag, watch the rollout
- Service with no endpoints: the selector does not match any pod labels
- Migration failure: /ready returns 503 and the pod stays out of endpoints
- Helm rollback restores chart configuration, not database migrations
- scripts/troubleshooting-check.py captures the whole transcript as evidence

Known Limits

- No authentication or RBAC — keep it local or add auth before exposing data
- Single PostgreSQL instance: no backups, failover or encryption policy
- Prometheus and Grafana storage is ephemeral; Postgres data is persistent
- Alerts have no notification destination configured
- AWS plan/apply/destroy proof requires a real account and is deferred

Key Learnings

- A deployment is only as trustworthy as the gates in front of it
- Readiness is a different question from liveness, and it changes routing
- Immutable SHA tags make production traceable back to a commit
- Metrics must be bounded: route templates, not IDs, keep dashboards sane
- Infrastructure as code is reviewable; console clicks are not
- One owner per deployment avoids Helm and Argo CD fighting each other